🌿 Git Cheat Sheet

Most useful commands — from first commit to advanced workflows

> **What is Git?** A distributed version control system. It tracks changes in your files locally.
> GitHub/GitLab/Bitbucket are just remote hosts for Git repos — `git` itself works entirely offline.

🔧 First-Time Setup

```
# Set your identity — used in every commit you make
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"

# Set default branch name to "main" (modern default, was "master")
git config --global init.defaultBranch main

# Set your preferred editor (for commit messages, rebase, etc.)
git config --global core.editor "code --wait"   # VS Code
git config --global core.editor vim

# Store credentials so you don't retype them (HTTPS users)
git config --global credential.helper store      # saves to disk permanently
git config --global credential.helper cache      # keeps in memory for 15 min

# View all your global config
git config --global --list

# View config for the current repo only (overrides global)
git config --list --local
```

📁 Creating & Cloning Repos

```
# Initialize a new repo in the current folder
git init

# Initialize with "main" as the default branch explicitly
git init -b main

# Clone a remote repo into a new folder
git clone https://github.com/user/repo.git

# Clone into a specific folder name
git clone https://github.com/user/repo.git my-folder

# Clone via SSH (requires SSH key set up)
```

```
git clone git@github.com:user/repo.git

# Clone only the latest snapshot — no full history (faster for large repos)
git clone --depth 1 https://github.com/user/repo.git
```

📸 The Three Stages

```
Working Directory  →  Staging Area (Index)  →  Local Repo  →  Remote Repo
   (edit files)         (git add)                (git commit)    (git push)
```

```
# See what's changed, staged, and untracked
git status

# Short status (compact view)
git status -s
```

➕ Staging (add)

```
# Stage a single file
git add README.md

# Stage multiple files
git add file1.py file2.py

# Stage everything in the current directory
git add .

# Stage all tracked files (skips new untracked files)
git add -u

# Interactively choose which changes within a file to stage (hunk by hunk)
git add -p README.md

# Unstage a file (keep changes in working dir)
git restore --staged README.md

# Unstage everything
git restore --staged .
```

💾 Committing

```
# Commit staged changes with a message
git commit -m "add login page"

# Stage all tracked changes AND commit in one step (skips untracked files)
git commit -am "fix typo in header"

# Open editor to write a longer commit message
git commit

# Amend the last commit (change message or add forgotten files)
# ⚠️ Only do this if you haven't pushed yet
git commit --amend -m "corrected commit message"
git commit --amend --no-edit    # keep the same message, just add staged changes

# Create an empty commit (useful to trigger CI)
git commit --allow-empty -m "trigger deploy"
```

🌿 Branches

```
# List local branches (* = current)
git branch

# List all branches including remote-tracking ones
git branch -a

# Create a new branch (stays on current branch)
git branch feature/login

# Switch to an existing branch
git switch feature/login         # modern (git 2.23+)
git checkout feature/login       # classic (still works)

# Create AND switch in one step
git switch -c feature/login      # modern
git checkout -b feature/login    # classic

# Rename current branch
git branch -m new-name

# Delete a branch (safe — refuses if unmerged)
git branch -d feature/login

# Force-delete a branch (unmerged changes will be lost)
git branch -D feature/login

# Delete a remote branch
git push origin --delete feature/login
```

Merging

```
# Merge a branch into the current branch
git merge feature/login

# Merge but always create a merge commit (no fast-forward)
git merge --no-ff feature/login

# Squash all commits from a branch into one staged change, then commit manually
git merge --squash feature/login
git commit -m "add login feature"

# Abort a merge that has conflicts
git merge --abort
```

Rebasing

> Rebase rewrites history — **never rebase branches others are working on.**

```
# Rebase current branch on top of main (replay your commits after main's commits)
git rebase main

# Interactive rebase — rewrite, squash, reorder the last N commits
git rebase -i HEAD~3              # opens editor for last 3 commits
# In the editor:
#   pick   → keep commit as-is
#   reword → keep but edit the message
#   squash → meld into the previous commit
#   drop   → delete the commit entirely

# Abort a rebase in progress
git rebase --abort

# Continue rebase after resolving conflicts
git rebase --continue
```

Remotes

```
# List remotes and their URLs
git remote -v

# Add a remote
git remote add origin https://github.com/user/repo.git

# Change remote URL (e.g. switch from HTTPS to SSH)
git remote set-url origin git@github.com:user/repo.git
```

```
# Rename a remote
git remote rename origin upstream

# Remove a remote
git remote remove upstream

# Show detailed info about a remote
git remote show origin
```

⬆️ Push & ⬇️ Pull

```
# Push current branch to its remote tracking branch
git push

# Push and set upstream tracking in one step (first push of a new branch)
git push -u origin feature/login

# Force push — overwrites remote history (⚠️ dangerous on shared branches)
git push --force

# Safer force push — only if remote hasn't changed since your last fetch
git push --force-with-lease

# Fetch remote changes without merging them
git fetch origin

# Fetch all remotes
git fetch --all

# Pull = fetch + merge
git pull

# Pull = fetch + rebase (cleaner linear history, recommended)
git pull --rebase

# Pull a specific branch
git pull origin main
```

📜 History & Diff

```
# View commit history
git log

# Compact one-line log
git log --oneline
```

```
# Graph view — shows branches and merges visually
git log --oneline --graph --all

# Log with stats (which files changed, how many lines)
git log --stat

# Show changes in a specific commit
git show abc1234

# Show changes made by a specific author
git log --author="Your Name"

# Show commits that changed a specific file
git log --follow -- path/to/file.py

# ── Diff ──────────────────────────────────────────────────────────────

# Diff between working dir and staging area (unstaged changes)
git diff

# Diff between staging area and last commit (staged changes)
git diff --staged

# Diff between two branches
git diff main..feature/login

# Diff a specific file between two commits
git diff abc1234 def5678 -- file.py
```

↩️ Undoing Things

```
# ── Discard unstaged changes in a file (⚠ unrecoverable) ──────────────
git restore file.py

# Discard ALL unstaged changes
git restore .

# ── Unstage a file (changes stay in working dir) ──────────────────────
git restore --staged file.py

# ── Undo the last commit, keep changes staged ─────────────────────────
git reset --soft HEAD~1

# Undo the last commit, keep changes in working dir (unstaged)
git reset --mixed HEAD~1        # this is the default

# Undo the last commit AND discard all changes (⚠ unrecoverable)
git reset --hard HEAD~1

# ── Revert a commit (safe — creates a new undo commit, doesn't rewrite history)
```

```
git revert abc1234              # use this on shared/public branches

# ── Recover a dropped commit or branch ──────────────────────────────
git reflog                      # shows every HEAD movement — your safety net
git checkout abc1234            # check out any commit from reflog
```

🗄️ Stash — Save Work Without Committing

```
# Stash current uncommitted changes (cleans working dir)
git stash

# Stash with a descriptive name
git stash push -m "half-done login form"

# Stash including untracked files
git stash push -u

# List all stashes
git stash list

# Apply the most recent stash (keeps stash in list)
git stash apply

# Apply and remove from stash list
git stash pop

# Apply a specific stash by index
git stash apply stash@{2}

# Drop a specific stash
git stash drop stash@{0}

# Clear all stashes
git stash clear
```

🏷️ Tags

```
# List all tags
git tag

# Create a lightweight tag at current commit
git tag v1.0.0

# Create an annotated tag (recommended — includes message, author, date)
git tag -a v1.0.0 -m "first stable release"

# Tag a specific past commit
```

```
git tag -a v0.9.0 abc1234 -m "beta"

# Push a tag to remote (tags are NOT pushed by default)
git push origin v1.0.0

# Push all local tags at once
git push origin --tags

# Delete a local tag
git tag -d v1.0.0

# Delete a remote tag
git push origin --delete v1.0.0

# Check out a tag (enters detached HEAD state)
git checkout v1.0.0
```

🍒 Cherry-pick — Copy a Commit to Another Branch

```
# Apply a specific commit from another branch onto the current branch
git cherry-pick abc1234

# Cherry-pick a range of commits
git cherry-pick abc1234..def5678

# Cherry-pick without auto-committing (stage the changes, let you edit)
git cherry-pick --no-commit abc1234

# Abort a cherry-pick with conflicts
git cherry-pick --abort
```

🔎 Find & Blame

```
# Search for a string across all tracked files
git grep "def login"

# Show who last changed each line of a file and in which commit
git blame file.py

# Show blame for specific lines only
git blame -L 10,25 file.py

# Binary search for the commit that introduced a bug
git bisect start
git bisect bad                  # current commit has the bug
git bisect good v1.0.0          # this tag was fine
# Git checks out a middle commit — test it, then:
```

```
git bisect good                 # or: git bisect bad
# Repeat until Git identifies the culprit commit
git bisect reset                # return to original branch when done
```

🧹 Cleanup

```
# Remove untracked files (dry run first — see what would be deleted)
git clean -n

# Remove untracked files (⚠️ permanent)
git clean -f

# Remove untracked files AND untracked directories
git clean -fd

# Remove ignored files too (e.g. build artifacts, __pycache__)
git clean -fdx

# Compress repo and remove unreachable objects (housekeeping)
git gc
```

📄 .gitignore Tips

```
# Force-ignore a file even if already tracked — remove from index only
git rm --cached file.py
git rm --cached -r __pycache__/

# Check why a file is being ignored (which rule matches)
git check-ignore -v file.py

# Temporarily stop tracking changes to a file (without adding to .gitignore)
git update-index --assume-unchanged config/local.py

# Resume tracking it
git update-index --no-assume-unchanged config/local.py
```

⚙️ Useful Config Shortcuts

```
# Set up handy aliases in ~/.gitconfig
git config --global alias.st    "status -s"
git config --global alias.co    "checkout"
git config --global alias.br    "branch"
git config --global alias.lg    "log --oneline --graph --all"
```

```
git config --global alias.undo  "reset --mixed HEAD~1"
git config --global alias.oops  "commit --amend --no-edit"

# Use them like normal commands:
git st
git lg
git undo
```

⚡ Quick Reference

Goal	Command
Start a repo	`git init`
Stage everything	`git add .`
Commit	`git commit -m "message"`
Push (first time)	`git push -u origin main`
Create + switch branch	`git switch -c feature/x`
Merge branch into main	`git switch main && git merge feature/x`
Stash changes	`git stash push -u -m "wip"`
Undo last commit (keep changes)	`git reset --soft HEAD~1`
Safe undo on public branch	`git revert <hash>`
See visual history	`git log --oneline --graph --all`
Who wrote this line?	`git blame file.py`
Find a bug's origin	`git bisect start`
Remove untracked files	`git clean -fd`
Safety net	`git reflog`

🚨 Rules to Live By

1. **Never force-push to `main`** — use `--force-with-lease` on feature branches only.
2. **Never rebase public/shared branches** — only rebase local branches not yet pushed.
3. **`git reflog` is your safety net** — almost nothing in Git is truly gone within 30 days.
4. **Commit early, commit often** — small commits are easy to revert; big ones are painful.
5. **Pull with `--rebase`** — keeps history linear and avoids pointless merge commits.
6. **Use `.gitignore` before first commit** — removing tracked files later is more work.
7. **Annotated tags over lightweight tags** — they carry metadata and show up in `git describe`.

Git reference docs: https://git-scm.com/docs